

Правительство Российской Федерации

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
«Высшая школа экономики»

ОТЧЕТ

к лабораторной работе №3 «The generation of pseudo random numbers»
по дисциплине «Методы программирования»

Выполнила студентка группы СКБ222

Маркова Варвара Владимировна

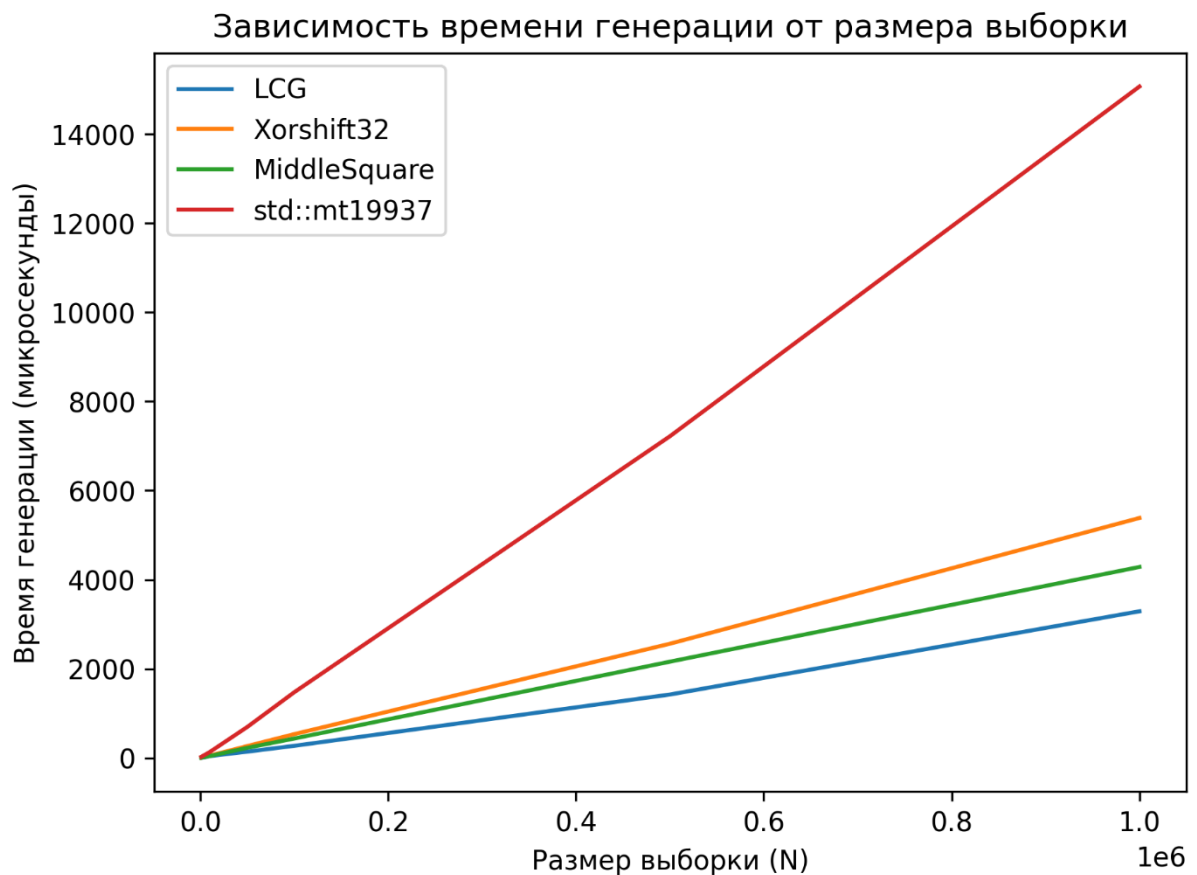
Москва 2025

Ссылка на исходный код программы в репозитории:

https://github.com/varvara0411/pm3_lab/tree/laba3

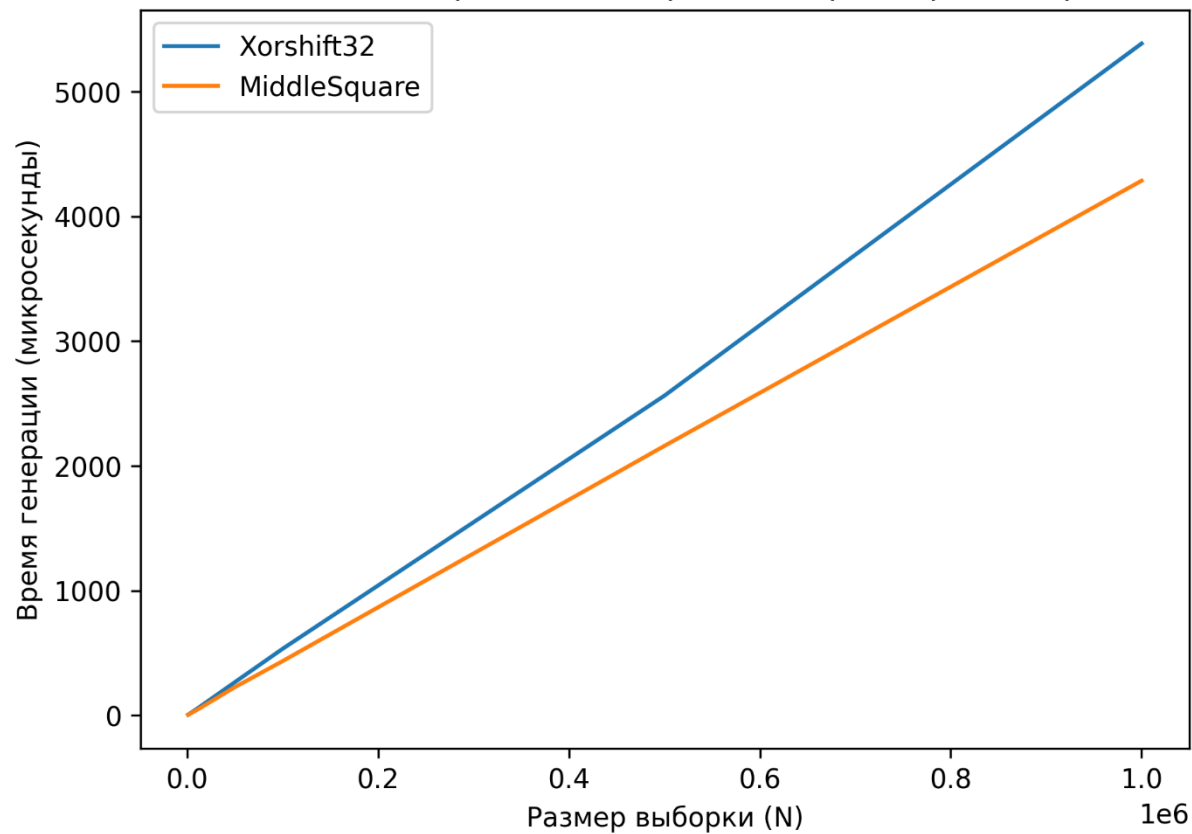
Графики времени работы четырех алгоритмов генерации псевдослучайных чисел.

Сначала построим график для всех четырех алгоритмов генерации псевдослучайных чисел:



Теперь для удобства сравним два наиболее близких по времени работы алгоритма генерации псевдослучайных чисел (MiddleSquare и Xorshift32):

Зависимость времени генерации от размера выборки



Lab 3

Generated by Doxygen 1.9.1

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 LCG Class Reference	5
3.1.1 Constructor & Destructor Documentation	5
3.1.1.1 LCG()	5
3.1.2 Member Function Documentation	6
3.1.2.1 next()	6
3.2 MiddleSquare Class Reference	6
3.2.1 Constructor & Destructor Documentation	6
3.2.1.1 MiddleSquare()	6
3.2.2 Member Function Documentation	7
3.2.2.1 next()	7
3.3 Xorshift32 Class Reference	7
3.3.1 Constructor & Destructor Documentation	7
3.3.1.1 Xorshift32()	7
3.3.2 Member Function Documentation	8
3.3.2.1 next()	8
4 File Documentation	9
4.1 func.cpp File Reference	9
4.1.1 Function Documentation	10
4.1.1.1 blockFrequencyTestP()	10
4.1.1.2 calculateStats()	10
4.1.1.3 chiSquareTestP()	11
4.1.1.4 cumulativeSumsTestP()	11
4.1.1.5 gammaQ()	11
4.1.1.6 generateBits()	12
4.1.1.7 incompleteGammaContinuedFrac()	12
4.1.1.8 incompleteGammaSeries()	12
4.1.1.9 longestRunOnesTestP()	13
4.1.1.10 monobitTestP()	13
4.1.1.11 performNISTTests()	13
4.1.1.12 Phi()	14
4.1.1.13 runsTestP()	14
4.2 func.h File Reference	15
4.2.1 Function Documentation	16
4.2.1.1 blockFrequencyTestP()	16
4.2.1.2 calculateStats()	18

4.2.1.3 chiSquareTestP()	18
4.2.1.4 cumulativeSumsTestP()	19
4.2.1.5 gammaQ()	19
4.2.1.6 generateBits()	20
4.2.1.7 incompleteGammaContinuedFrac()	20
4.2.1.8 incompleteGammaSeries()	21
4.2.1.9 longestRunOnesTestP()	22
4.2.1.10 monobitTestP()	22
4.2.1.11 performNISTTests()	23
4.2.1.12 Phi()	23
4.2.1.13 runsTestP()	24
4.3 main.cpp File Reference	24
4.3.1 Detailed Description	25
4.3.2 Function Documentation	25
4.3.2.1 main()	25
4.4 plt.py File Reference	26
Index	27

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

LCG	Linear Congruential Generator (LCG) random number generator	5
MiddleSquare	Middle Square random number generator	6
Xorshift32	Xorshift32 random number generator	7

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

func.cpp	Implementation of random number generators and statistical tests for randomness analysis . .	9
func.h	Declaration of random number generators (LCG , Xorshift32 , MiddleSquare) and statistical test functions for randomness analysis	15
main.cpp	Random Number Generator Testing Application	24
plt.py	Performance comparison visualization for random number generators	26

Chapter 3

Class Documentation

3.1 LCG Class Reference

Linear Congruential Generator ([LCG](#)) random number generator.

```
#include <func.h>
```

Public Member Functions

- [LCG](#) (uint32_t seed)
Generates the next random number in sequence.
- uint32_t [next](#) ()
Generates the next random number in sequence.

3.1.1 Constructor & Destructor Documentation

3.1.1.1 LCG()

```
LCG::LCG (  
    uint32_t seed )
```

Constructs a Linear Congruential Generator with given seed.

Returns

Next 32-bit unsigned random number

Parameters

<i>seed</i>	Initial seed value
-------------	--------------------

3.1.2 Member Function Documentation

3.1.2.1 next()

```
uint32_t LCG::next ( )
```

Generates next random number using [LCG](#) algorithm.

Returns

Next 32-bit unsigned random number
32-bit unsigned random number

The documentation for this class was generated from the following files:

- [func.h](#)
- [func.cpp](#)

3.2 MiddleSquare Class Reference

Middle Square random number generator.

```
#include <func.h>
```

Public Member Functions

- [MiddleSquare](#) (uint32_t seed)
Constructor that initializes the generator with a seed.
- uint32_t [next](#) ()
Generates the next random number using middle-square method.

3.2.1 Constructor & Destructor Documentation

3.2.1.1 MiddleSquare()

```
MiddleSquare::MiddleSquare (  
    uint32_t seed )
```

Constructs a Middle Square generator with given seed.

Parameters

<i>seed</i>	Initial seed value
-------------	--------------------

3.2.2 Member Function Documentation

3.2.2.1 next()

```
uint32_t MiddleSquare::next ( )
```

Generates next random number using Middle Square method.

Returns

Next 32-bit unsigned random number
32-bit unsigned random number

The documentation for this class was generated from the following files:

- [func.h](#)
- [func.cpp](#)

3.3 Xorshift32 Class Reference

[Xorshift32](#) random number generator.

```
#include <func.h>
```

Public Member Functions

- [Xorshift32](#) (uint32_t seed)
Constructor that initializes the generator with a seed.
- uint32_t [next](#) ()
Generates the next random number using xorshift algorithm.

3.3.1 Constructor & Destructor Documentation

3.3.1.1 Xorshift32()

```
Xorshift32::Xorshift32 (
    uint32_t seed )
```

Constructs a [Xorshift32](#) generator with given seed.

Parameters

<i>seed</i>	Initial seed value
-------------	--------------------

3.3.2 Member Function Documentation

3.3.2.1 next()

```
uint32_t Xorshift32::next ( )
```

Generates next random number using Xorshift algorithm.

Returns

Next 32-bit unsigned random number

32-bit unsigned random number

The documentation for this class was generated from the following files:

- [func.h](#)
- [func.cpp](#)

Chapter 4

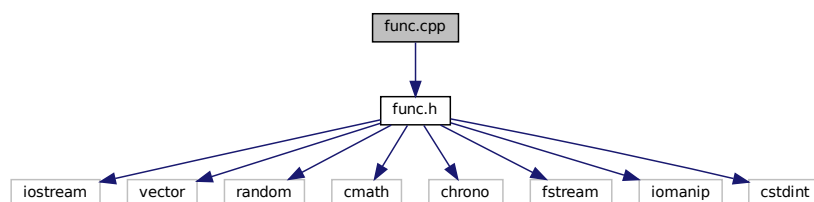
File Documentation

4.1 func.cpp File Reference

Implementation of random number generators and statistical tests for randomness analysis.

```
#include "func.h"
```

Include dependency graph for func.cpp:



Functions

- double [incompleteGammaSeries](#) (double a, double x)
Calculates incomplete gamma function using series expansion.
- double [incompleteGammaContinuedFrac](#) (double a, double x)
Calculates incomplete gamma function using continued fraction.
- double [gammaQ](#) (double a, double x)
Computes regularized upper incomplete gamma function $Q(a,x)$
- double [Phi](#) (double x)
Computes standard normal cumulative distribution function.
- double [monobitTestP](#) (const std::vector< int > &bits)
Performs the monobit test on a binary sequence (NIST SP 800-22)
- double [blockFrequencyTestP](#) (const std::vector< int > &bits, int M=128)
Performs the block frequency test on a binary sequence (NIST SP 800-22)
- double [runsTestP](#) (const std::vector< int > &bits)
Performs the runs test on a binary sequence (NIST SP 800-22)
- double [longestRunOnesTestP](#) (const std::vector< int > &bits)

- Performs the longest run of ones test on a binary sequence (NIST SP 800-22)*
- double [cumulativeSumsTestP](#) (const std::vector< int > &bits)
- Performs the cumulative sums test on a binary sequence (NIST SP 800-22)*
- void [calculateStats](#) (const std::vector< uint32_t > &values, double &mean, double &stddev, double &cov)
- Calculates basic statistics for a set of values.*
- double [chiSquareTestP](#) (const std::vector< uint32_t > &values)
- Performs chi-square goodness-of-fit test on random numbers.*
- std::vector< int > [generateBits](#) (const std::vector< uint32_t > &values)
- Converts 32-bit values to a sequence of bits.*
- void [performNISTTests](#) (const std::vector< int > &bits, double &p1, double &p2, double &p3, double &p4, double &p5)
- Performs battery of NIST statistical tests on bit sequence.*

4.1.1 Function Documentation

4.1.1.1 blockFrequencyTestP()

```
double blockFrequencyTestP (
    const std::vector< int > & bits,
    int M = 128 )
```

Performs the block frequency test on a binary sequence.

Parameters

<i>bits</i>	Vector of bits (0s and 1s) to test
<i>M</i>	Block size (default = 128)

Returns

P-value indicating the probability of randomness

Tests the proportion of ones within M-bit blocks to verify they are approximately 50%

4.1.1.2 calculateStats()

```
void calculateStats (
    const std::vector< uint32_t > & values,
    double & mean,
    double & stddev,
    double & cov )
```

Parameters

	<i>values</i>	Input vector of 32-bit unsigned integers
out	<i>mean</i>	Calculated mean value
out	<i>stddev</i>	Calculated standard deviation
out	<i>cov</i>	Calculated coefficient of variation (stddev/mean)

4.1.1.3 chiSquareTestP()

```
double chiSquareTestP (
    const std::vector< uint32_t > & values )
```

Performs chi-square goodness-of-fit test.

Parameters

<i>values</i>	Vector of 32-bit unsigned integers to test
---------------	--

Returns

P-value indicating the probability of uniform distribution

Tests if the numbers are uniformly distributed across the range

4.1.1.4 cumulativeSumsTestP()

```
double cumulativeSumsTestP (
    const std::vector< int > & bits )
```

Performs the cumulative sums test on a binary sequence.

Parameters

<i>bits</i>	Vector of bits (0s and 1s) to test
-------------	------------------------------------

Returns

P-value indicating the probability of randomness

Tests the maximum deviation of the cumulative sum of the sequence from zero

4.1.1.5 gammaQ()

```
double gammaQ (
    double a,
    double x )
```

Calculates the regularized upper incomplete gamma function $Q(a,x)$

Parameters

<i>a</i>	Shape parameter
<i>x</i>	Upper limit of integration

Returns

Value of $Q(a,x)$

4.1.1.6 generateBits()

```
std::vector<int> generateBits (
    const std::vector< uint32_t > & values )
```

Parameters

<i>values</i>	Vector of 32-bit unsigned integers
---------------	------------------------------------

Returns

Vector of bits (0s and 1s) extracted from the input values

Extracts all 32 bits from each input value (MSB first)

4.1.1.7 incompleteGammaContinuedFrac()

```
double incompleteGammaContinuedFrac (
    double a,
    double x )
```

Parameters

<i>a</i>	Shape parameter
<i>x</i>	Upper limit of integration

Returns

Computed value of incomplete gamma function

4.1.1.8 incompleteGammaSeries()

```
double incompleteGammaSeries (
    double a,
    double x )
```

Parameters

<i>a</i>	Shape parameter
<i>x</i>	Upper limit of integration

Returns

Computed value of incomplete gamma function

4.1.1.9 longestRunOnesTestP()

```
double longestRunOnesTestP (
    const std::vector< int > & bits )
```

Performs the longest run of ones test on a binary sequence.

Parameters

<i>bits</i>	Vector of bits (0s and 1s) to test
-------------	------------------------------------

Returns

P-value indicating the probability of randomness

Tests the longest run of ones within M-bit blocks of the sequence

4.1.1.10 monobitTestP()

```
double monobitTestP (
    const std::vector< int > & bits )
```

Performs the monobit test on a binary sequence.

Parameters

<i>bits</i>	Vector of bits (0s and 1s) to test
-------------	------------------------------------

Returns

P-value indicating the probability of randomness (values ≥ 0.01 suggest randomness)

Tests the proportion of ones and zeros in the sequence to verify they are approximately equal

4.1.1.11 performNISTTests()

```
void performNISTTests (
    const std::vector< int > & bits,
    double & p1,
    double & p2,
    double & p3,
    double & p4,
    double & p5 )
```

Performs a battery of NIST statistical tests on a bit sequence.

Parameters

	<i>bits</i>	Input bit sequence to test
out	<i>p1</i>	P-value from monobit test
out	<i>p2</i>	P-value from block frequency test
out	<i>p3</i>	P-value from runs test
out	<i>p4</i>	P-value from longest run of ones test
out	<i>p5</i>	P-value from cumulative sums test

4.1.1.12 Phi()

```
double Phi (
    double x )
```

Calculates the standard normal cumulative distribution function.

Parameters

<i>x</i>	Input value
----------	-------------

Returns

Probability that a random variable is less than or equal to x

4.1.1.13 runsTestP()

```
double runsTestP (
    const std::vector< int > & bits )
```

Performs the runs test on a binary sequence.

Parameters

<i>bits</i>	Vector of bits (0s and 1s) to test
-------------	------------------------------------

Returns

P-value indicating the probability of randomness

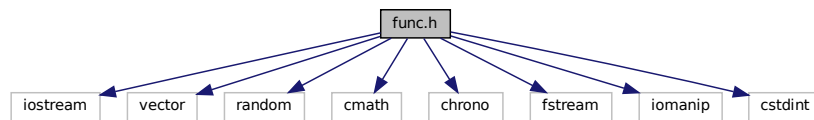
Tests the total number of runs (both 0-runs and 1-runs) in the sequence

4.2 func.h File Reference

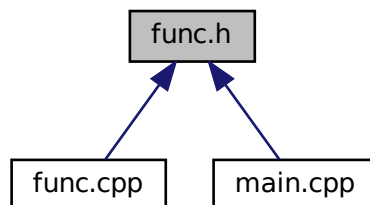
Declaration of random number generators ([LCG](#), [Xorshift32](#), [MiddleSquare](#)) and statistical test functions for randomness analysis.

```
#include <iostream>
#include <vector>
#include <random>
#include <cmath>
#include <chrono>
#include <fstream>
#include <iomanip>
#include <cstdlib>
```

Include dependency graph for func.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [LCG](#)
Linear Congruential Generator (LCG) random number generator.
- class [Xorshift32](#)
Xorshift32 random number generator.
- class [MiddleSquare](#)
Middle Square random number generator.

Functions

- double [incompleteGammaSeries](#) (double a, double x)
Calculates incomplete gamma function using series expansion.
- double [incompleteGammaContinuedFrac](#) (double a, double x)
Calculates incomplete gamma function using continued fraction.
- double [gammaQ](#) (double a, double x)
Calculates the regularized upper incomplete gamma function $Q(a,x)$
- double [Phi](#) (double x)
Calculates the standard normal cumulative distribution function.
- double [monobitTestP](#) (const std::vector< int > &bits)
Performs the monobit test on a binary sequence.
- double [blockFrequencyTestP](#) (const std::vector< int > &bits, int M)
Performs the block frequency test on a binary sequence.
- double [runsTestP](#) (const std::vector< int > &bits)
Performs the runs test on a binary sequence.
- double [longestRunOnesTestP](#) (const std::vector< int > &bits)
Performs the longest run of ones test on a binary sequence.
- double [cumulativeSumsTestP](#) (const std::vector< int > &bits)
Performs the cumulative sums test on a binary sequence.
- void [calculateStats](#) (const std::vector< uint32_t > &values, double &mean, double &stddev, double &cov)
Calculates basic statistics for a set of values.
- double [chiSquareTestP](#) (const std::vector< uint32_t > &values)
Performs chi-square goodness-of-fit test.
- std::vector< int > [generateBits](#) (const std::vector< uint32_t > &values)
Converts 32-bit values to a sequence of bits.
- void [performNISTTests](#) (const std::vector< int > &bits, double &p1, double &p2, double &p3, double &p4, double &p5)
Performs a battery of NIST statistical tests on a bit sequence.

4.2.1 Function Documentation

4.2.1.1 blockFrequencyTestP()

```
double blockFrequencyTestP (
    const std::vector< int > & bits,
    int M = 128 )
```

Parameters

<i>bits</i>	Vector of bits (0s and 1s)
<i>M</i>	Block size

Returns

P-value from the test

Performs the block frequency test on a binary sequence.

Parameters

<i>bits</i>	Vector of bits (0s and 1s) to test
<i>M</i>	Block size (default = 128)

Returns

P-value indicating the probability of randomness

Tests the proportion of ones within M-bit blocks to verify they are approximately 50%

4.2.1.2 calculateStats()

```
void calculateStats (
    const std::vector< uint32_t > & values,
    double & mean,
    double & stddev,
    double & cov )
```

Parameters

	<i>values</i>	Input values
out	<i>mean</i>	Calculated mean
out	<i>stddev</i>	Calculated standard deviation
out	<i>cov</i>	Calculated coefficient of variation
	<i>values</i>	Input vector of 32-bit unsigned integers
out	<i>mean</i>	Calculated mean value
out	<i>stddev</i>	Calculated standard deviation
out	<i>cov</i>	Calculated coefficient of variation (stddev/mean)

4.2.1.3 chiSquareTestP()

```
double chiSquareTestP (
    const std::vector< uint32_t > & values )
```

Parameters

<i>values</i>	Input values to test
---------------	----------------------

Returns

P-value from the test

Performs chi-square goodness-of-fit test.

Parameters

<i>values</i>	Vector of 32-bit unsigned integers to test
---------------	--

Returns

P-value indicating the probability of uniform distribution

Tests if the numbers are uniformly distributed across the range

4.2.1.4 cumulativeSumsTestP()

```
double cumulativeSumsTestP (  
    const std::vector< int > & bits )
```

Parameters

<i>bits</i>	Vector of bits (0s and 1s)
-------------	----------------------------

Returns

P-value from the test

Performs the cumulative sums test on a binary sequence.

Parameters

<i>bits</i>	Vector of bits (0s and 1s) to test
-------------	------------------------------------

Returns

P-value indicating the probability of randomness

Tests the maximum deviation of the cumulative sum of the sequence from zero

4.2.1.5 gammaQ()

```
double gammaQ (  
    double a,  
    double x )
```

Parameters

<i>a</i>	Shape parameter
<i>x</i>	Upper limit of integration

Returns

Value of $Q(a,x)$

Calculates the regularized upper incomplete gamma function $Q(a,x)$

Parameters

<i>a</i>	Shape parameter
<i>x</i>	Upper limit of integration

Returns

Value of $Q(a,x)$

4.2.1.6 generateBits()

```
std::vector<int> generateBits (
    const std::vector< uint32_t > & values )
```

Parameters

<i>values</i>	Input values
---------------	--------------

Returns

Vector of bits (0s and 1s)

Parameters

<i>values</i>	Vector of 32-bit unsigned integers
---------------	------------------------------------

Returns

Vector of bits (0s and 1s) extracted from the input values

Extracts all 32 bits from each input value (MSB first)

4.2.1.7 incompleteGammaContinuedFrac()

```
double incompleteGammaContinuedFrac (
    double a,
    double x )
```

Parameters

a	Shape parameter
x	Upper limit of integration

Returns

Value of the incomplete gamma function

Parameters

a	Shape parameter
x	Upper limit of integration

Returns

Computed value of incomplete gamma function

4.2.1.8 incompleteGammaSeries()

```
double incompleteGammaSeries (
    double a,
    double x )
```

Parameters

a	Shape parameter
x	Upper limit of integration

Returns

Value of the incomplete gamma function

Parameters

a	Shape parameter
x	Upper limit of integration

Returns

Computed value of incomplete gamma function

4.2.1.9 longestRunOnesTestP()

```
double longestRunOnesTestP (
    const std::vector< int > & bits )
```

Parameters

<i>bits</i>	Vector of bits (0s and 1s)
-------------	----------------------------

Returns

P-value from the test

Performs the longest run of ones test on a binary sequence.

Parameters

<i>bits</i>	Vector of bits (0s and 1s) to test
-------------	------------------------------------

Returns

P-value indicating the probability of randomness

Tests the longest run of ones within M-bit blocks of the sequence

4.2.1.10 monobitTestP()

```
double monobitTestP (
    const std::vector< int > & bits )
```

Parameters

<i>bits</i>	Vector of bits (0s and 1s)
-------------	----------------------------

Returns

P-value from the test

Performs the monobit test on a binary sequence.

Parameters

<i>bits</i>	Vector of bits (0s and 1s) to test
-------------	------------------------------------

Returns

P-value indicating the probability of randomness (values ≥ 0.01 suggest randomness)

Tests the proportion of ones and zeros in the sequence to verify they are approximately equal

4.2.1.11 performNISTTests()

```
void performNISTTests (
    const std::vector< int > & bits,
    double & p1,
    double & p2,
    double & p3,
    double & p4,
    double & p5 )
```

Parameters

	<i>bits</i>	Input bit sequence to test
out	<i>p1</i>	P-value from monobit test
out	<i>p2</i>	P-value from block frequency test
out	<i>p3</i>	P-value from runs test
out	<i>p4</i>	P-value from longest run of ones test
out	<i>p5</i>	P-value from cumulative sums test

Performs a battery of NIST statistical tests on a bit sequence.

Parameters

	<i>bits</i>	Input bit sequence to test
out	<i>p1</i>	P-value from monobit test
out	<i>p2</i>	P-value from block frequency test
out	<i>p3</i>	P-value from runs test
out	<i>p4</i>	P-value from longest run of ones test
out	<i>p5</i>	P-value from cumulative sums test

4.2.1.12 Phi()

```
double Phi (
    double x )
```

Parameters

<i>x</i>	Input value
----------	-------------

Returns

Probability that a random variable is less than or equal to x

Calculates the standard normal cumulative distribution function.

Parameters

<i>x</i>	Input value
----------	-------------

Returns

Probability that a random variable is less than or equal to *x*

4.2.1.13 runsTestP()

```
double runsTestP (
    const std::vector< int > & bits )
```

Parameters

<i>bits</i>	Vector of bits (0s and 1s)
-------------	----------------------------

Returns

P-value from the test

Performs the runs test on a binary sequence.

Parameters

<i>bits</i>	Vector of bits (0s and 1s) to test
-------------	------------------------------------

Returns

P-value indicating the probability of randomness

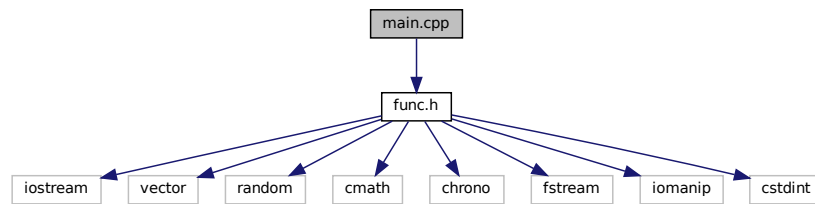
Tests the total number of runs (both 0-runs and 1-runs) in the sequence

4.3 main.cpp File Reference

Random Number Generator Testing Application.

```
#include "func.h"
```

Include dependency graph for main.cpp:



Functions

- `int main ()`
Main function for random number generator testing.

4.3.1 Detailed Description

This program evaluates and compares four pseudo-random number generators:

1. Linear Congruential Generator ([LCG](#))
2. [Xorshift32](#)
3. Middle Square Method
4. Mersenne Twister (`std::mt19937`)

The evaluation includes statistical tests and performance benchmarks.

4.3.2 Function Documentation

4.3.2.1 main()

```
int main ( )
```

Returns

Exit code (0 for success)

The program performs the following operations:

1. Generates test sequences for each PRNG algorithm
2. Saves sequences to files for further analysis
3. Conducts statistical analysis:
 - Basic statistics (mean, standard deviation, coefficient of variation)
 - Uniformity test (chi-square)
 - Randomness tests (NIST test suite)
4. Measures and compares generation performance
5. Outputs results in both human-readable and CSV formats

4.4 plt.py File Reference

Performance comparison visualization for random number generators.

Variables

- **plt.df** = `pd.read_csv('times.csv')`
- **plt.label**
- **plt.dpi**
- **plt.bbox_inches**

Index

blockFrequencyTestP
 func.cpp, 10
 func.h, 16

calculateStats
 func.cpp, 10
 func.h, 18

chiSquareTestP
 func.cpp, 11
 func.h, 18

cumulativeSumsTestP
 func.cpp, 11
 func.h, 19

func.cpp, 9
 blockFrequencyTestP, 10
 calculateStats, 10
 chiSquareTestP, 11
 cumulativeSumsTestP, 11
 gammaQ, 11
 generateBits, 12
 incompleteGammaContinuedFrac, 12
 incompleteGammaSeries, 12
 longestRunOnesTestP, 13
 monobitTestP, 13
 performNISTTests, 13
 Phi, 14
 runsTestP, 14

func.h, 15
 blockFrequencyTestP, 16
 calculateStats, 18
 chiSquareTestP, 18
 cumulativeSumsTestP, 19
 gammaQ, 19
 generateBits, 20
 incompleteGammaContinuedFrac, 20
 incompleteGammaSeries, 21
 longestRunOnesTestP, 21
 monobitTestP, 22
 performNISTTests, 22
 Phi, 23
 runsTestP, 24

gammaQ
 func.cpp, 11
 func.h, 19

generateBits
 func.cpp, 12
 func.h, 20

incompleteGammaContinuedFrac
 func.cpp, 12
 func.h, 20

incompleteGammaSeries
 func.cpp, 12
 func.h, 21

LCG, 5
 LCG, 5
 next, 6

longestRunOnesTestP
 func.cpp, 13
 func.h, 21

main
 main.cpp, 25

main.cpp, 24
 main, 25

MiddleSquare, 6
 MiddleSquare, 6
 next, 7

monobitTestP
 func.cpp, 13
 func.h, 22

next
 LCG, 6
 MiddleSquare, 7
 Xorshift32, 8

performNISTTests
 func.cpp, 13
 func.h, 22

Phi
 func.cpp, 14
 func.h, 23

plt.py, 26

runsTestP
 func.cpp, 14
 func.h, 24

Xorshift32, 7
 next, 8
 Xorshift32, 7